PROGRAMMING NEURAL NETWORKS

Your First Deep Learning Project in Python with Keras

- We will learn how to build a neural network in Python with keras.
- Dataset: Pima-Indians-Diabetes
- Input Variables (X):
  - Number of times pregnant
  - Plasma glucose concentration 2 hours in an oral glucose tolerance test
  - Diastolic blood pressure (mm Hg)
  - Triceps skin fold thickness (mm)
  - 2-Hour serum insulin (mu U/ml)
  - Body mass index (weight in kg/(height in m)^2)
  - Diabetes pedigree function
  - Age (years)
- Output Variable (y): Class variable (0 or 1)

Step 1 load Data

```
# first neural network with keras tutorial

from keras.models import Sequential

from keras.layers import Dense

import pandas

from sklearn import model_selection

# load the dataset

dataset = pandas.read_csv('C:\\pima-indians-diabetes.csv')

# split into input (X) and output (y) variables

array = dataset.values

X = array[:,0:8]

y = array[:,8]

X_train, X_validation, Y_train, Y_validation = model_selection.train_test_split(X, y,
test_size=0.15, random_state=7)
```

Step#2: Define Keras Model

Models in Keras are defined as a sequence of layers.

We create a Sequential model and add layers one at a time until we are happy with our network architecture.

The first thing to get right is to ensure the input layer has the right number of input features. This can be specified when creating the first layer with the **input_dim** argument and setting it to 8 for the 8 input variables.

This is a very hard question. There are heuristics that we can use and often the best network structure is found through a process of trial and error experimentation. Generally, you need a network large enough to capture the structure of the problem.

In this example, we will use a fully-connected network structure with three layers.

```
# define the keras model
model = Sequential()
```

```
model.add(Dense(12, input_dim=8, activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
```

Step#3: Compile Keras Model

```
# compile the keras model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

Step#4: Fit Keras Model

Training occurs over epochs and each epoch is split into batches.

- Epoch: One pass through all of the rows in the training dataset.
- Batch: One or more samples considered by the model within an epoch before weights are updated.

For this problem, we will run for a small number of epochs (150) and use a relatively small batch size of 10.

These configurations can be chosen experimentally by trial and error. We want to train the model enough so that it learns a good (or good enough) mapping of rows of input data to the output classification.

```
# fit the keras model on the dataset
model.fit(X_train, Y_train, epochs=150, batch_size=10)
```

Step#5: Evaluate Keras Model

```
print ('**************************')

# evaluate the keras model

loss,accuracy = model.evaluate(X_validation, Y_validation)

print('Accuracy: %.2f' % (accuracy*100))

print (loss)
```

Neural networks are a stochastic algorithm.

Step #6: Make predictions

```
# make class predictions with the model

predictions = model.predict_classes(X_validation)

# summarize the first 25 cases

for i in range(25):

        print( X_validation[i], predictions[i], Y_validation[i])
```